トークン

DJ の同意で受け取った `code` を access token に交換し、期限前に更新し、必要なら失効させます。ボット API を呼ぶには、まずここでトークンを受け取ります。

RFC 6749(OAuth 2.0)と RFC 7009(失効)に従います。

共通ルール

```
POST $SPOON_BASE_URL/v1/oauth/token
POST $SPOON_BASE_URL/v1/oauth/revoke
```

名前	位置	必須	説明
`Content-Type`	header	✔	`application/x-www-form-urlencoded` です。**JSON ではありません。**

クライアント認証

2 つの方法があり、**Basic ヘッダーが優先** されます。

```
Authorization: Basic base64(client_id:client_secret)
```

ヘッダーがなければ form の `client_id`・`client_secret` にフォールバックします。どちらもなければ `401 invalid_client` です。`client_id` は **UUID 形式** である必要があり、そうでなければ同じく `401 invalid_client` です。

`401` のレスポンスには `WWW-Authenticate: Basic realm="oauth"` ヘッダーが付きます。

Client Secret を使う呼び出しなので、サーバーからのみ呼びます。ブラウザやモバイルアプリの配布物から呼ぶと Secret が漏れます。

トークンを発行する

同意後に受け取った `code` をトークンのペアに交換します。

```
POST /v1/oauth/token
```

form フィールド	必須	説明
`grant_type`	✔	`authorization_code`
`code`	✔	同意後に受け取った 1 回限りの値です。**60 秒** 以内に使います。
`redirect_uri`	✔	同意リクエストで使ったものと **一致** している必要があります。

```
curl -X POST "$SPOON_BASE_URL/v1/oauth/token" \
  -u "{Client ID}:{Client Secret}" \
  -d "grant_type=authorization_code" \
  -d "code={受け取った code}" \
  -d "redirect_uri={ログインリダイレクト URL}"
```

```ts
// refresh.ts — node --experimental-strip-types refresh.ts

import { writeFile } from 'node:fs/promises';

interface Token {
  access_token: string;
  token_type: string;
  expires_in: number;
  refresh_token: string;
  scope: string;
}

interface OauthError {
  error: string;
  error_description: string;
}

const basic = Buffer.from(
  `${process.env.SPOON_CLIENT_ID}:${process.env.SPOON_CLIENT_SECRET}`,
).toString('base64');
```

```
const res = await fetch(`${process.env.SPOON_BASE_URL}/v1/oauth/token`, {
  method: 'POST',
  headers: {
    Authorization: `Basic ${basic}`,
    'Content-Type': 'application/x-www-form-urlencoded',
  },
  body: new URLSearchParams({
    grant_type: 'refresh_token',
    refresh_token: process.env.SPOON_REFRESH_TOKEN ?? '',
  }),
});

if (!res.ok) {
  const { error, error_description } = (await res.json()) as OauthError;
  throw new Error(`更新に失敗しました ${error}: ${error_description}`);
}

const token = (await res.json()) as Token;
await writeFile('token.json', JSON.stringify(token, null, 2));    // 新しい refresh_token を必ず保存
console.log(`scope: ${token.scope}`);                              // 更新のたびに確認
```

```
export SPOON_CLIENT_ID="{Client ID}"
export SPOON_CLIENT_SECRET="{Client Secret}"
export SPOON_REFRESH_TOKEN="{受け取った refresh_token}"
node --experimental-strip-types refresh.ts
```

> 🔥 `code` を 2 回使うと、その連携のトークンがすべて失効します
>
> 同じ `code` で 2 回リクエストすると盗用のシグナルとみなし、**その連携で発行された トークンをすべて失効**させます。レスポンスは最初の失敗と同じ `invalid_grant` なので 見た目では区別できませんが、結果ははるかに重いです。
>
> ネットワークエラーでレスポンスを受け取れなかったときに、**同じ `code` で再試行しないで ください**。その `code` は捨てて、同意からやり直します。

トークンを更新する

access token が失効する前に新しいペアを受け取ります。

```
POST /v1/oauth/token
```

form フィールド	必須	説明
`grant_type`	✔	`refresh_token`
`refresh_token`	✔	最後に受け取った refresh token です。

```
curl -X POST "$SPOON_BASE_URL/v1/oauth/token" \
  -u "{Client ID}:{Client Secret}" \
  -d "grant_type=refresh_token" \
  -d "refresh_token={受け取った refresh_token}"
```

> 🔥 **REFRESH TOKEN はローテーションします — レスポンスの新しい値を必ず保存します**
>
> 更新すると **古い refresh token は直ちに失効し**、新しい access・refresh のペアが返ります。レスポンスの新しい `refresh_token` を保存しないと、**次の更新で `400 invalid_grant` になり ボットが停止します**。更新自体は成功していて、その次だけが失敗する形なので原因を 見つけにくいです。

古い access token は一緒に失効させず、自然な期限切れに任せます。そのため更新直後の 短い間は、2 つの access token がどちらも有効です。

更新のたびに `scope` を確認します。レスポンスの `scope` は **その時点の DJ の同意** に基づきます。DJ が同意を減らすと次の 更新で自動的に反映されます。発行時のスコープをずっと持っている前提でいると、すでに 外れた権限で API を呼んで `403` を受け取ります。

更新の時刻は余裕をもって取ります

期限直前に更新すると、ワーカーが少し止まって戻ったときにはもう間に合いません。**期限の 5 分前**。のように大きめのマージンを取り、更新結果は再起動しても残るように 保存します。メモリにだけ置くと、プロセスを立ち上げ直すたびに DJ から同意を取り直す ことになります。

事前に更新しても、**すでに開いているイベントストリームは接続に使ったトークンの失効時刻に終わります。** `TOKEN_EXPIRED` を受け取ったら、更新してすぐに接続し直します。事前の更新は、チャット送信などの REST 呼び出しには引き続き必要です。

レスポンス(発行・更新 共通)

フィールド名は snake_case 固定です。

```
{
  "access_token": "at_3f9c1e7b8d2a4056...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "rt_7b2e5a9c4f10d386...",
  "scope": "events.chat chat.send"
}
```

フィールド	型	説明
`access_token`	string	ボット API の呼び出しに使う値です。
`token_type`	string	常に `Bearer` です。
`expires_in`	number	access token の残り有効期間です。秒単位で、既定は **1 時間 (3600)** です。
`refresh_token`	string	次の更新に使う値です。既定の有効期間は **30 日** です。
`scope`	string	DJ が同意した権限です。**スペース区切り** です。

以降、すべてのボット API はこう呼びます。

```
Authorization: Bearer {access_token}
```

トークンを失効させる

発行されたトークンを無効にします。

```
POST /v1/oauth/revoke
```

form フィールド	必須	説明
`token`	✔	access token でも refresh token でも構いません。

form フィールド	必須	説明
`token_type_hint`		受け取りますが **無視します** — トークン文字列から直接判別します。

```
curl -X POST "$SPOON_BASE_URL/v1/oauth/revoke" \
  -u "{Client ID}:{Client Secret}" \
  -d "token={失効させるトークン}"
```

成功するとボディなしで `200` です。

- **存在しないトークンや他人のトークンを送っても `200` です。**トークンの存在有無を 外部に知らせないためのルールです(RFC 7009)。したがって `200` は「そのトークンが あった」を意味しません。
- **refresh token を失効させると、同じ連携の access token も一緒に失効** します。

失効は「連携解除」ではありません。DJ の同意(grant)はそのまま残ります。トークンだけが無効になるので、改めて同意を 得なくても 同意画面 を経由して新しい トークンを受け取れます。

DJ が連携そのものを解除するのは別のことです。その場合はトークンが無効になり、再同意も必要になります。

エラー

形式が他のボット API と異なります。ボットランタイム API は `{"status", "detailCode", "message"}` ですが、OAuth エンドポイントは RFC 6749 §5.2 の形式です。エラーコード の `detailCode` はここには出てきません。

```
{
  "error": "invalid_grant",
  "error_description": "invalid, expired or already used code"
}
```

`error`	HTTP	いつ	どうするか
`invalid_request`	`400`	必須 form フィールドが不足しています (`grant_type`・`code`・`redirect_uri`・`refresh_token`・`token`)。	リクエストの形式を直します。

error	HTTP	いつ	どうするか
invalid_client	401	クライアント認証に失敗、client_id の形式が不正、またはアプリが停止中です。	資格情報とアプリの状態を確認します。
invalid_grant	400	code が失効・使用済み、他のアプリに発行された code、redirect_uri の不一致、refresh token が無効・失効しています。	code は同意からやり直します。refresh が無効なら DJ に再連携を案内します。
unsupported_grant_type	400	authorization_code・refresh_token 以外の値です。	どちらかに直します。

error_description で分岐しないでください。人が読むための説明で、文言が変わる 可能性があります。分岐は error で行います。

invalid_scope はここではなく 同意のステップ で返ります。

関連ドキュメント

- エンドポイント一覧 — トークンで呼び出せるエンドポイントです。
- ユーザーの同意を得る — code を受け取る前のステップです。
- ボットを作る — 最初から最後まで動く最小のサンプルです。
- 権限を選ぶ — scope に入る値です。